

Capítulo 1

Marco teórico

Este capítulo reúne los fundamentos que sostienen el detector del capítulo siguiente: cómo se convierte un texto en números, qué modelos clasifican esos números, cómo se validan y con qué métricas se miden. Cada concepto se presenta una vez y en su forma general; su aplicación concreta, con los datos y las cifras del problema, se desarrolla en el caso práctico.

1.1. Aprendizaje automático

Un modelo de aprendizaje automático es un programa que aprende a reconocer patrones a partir de ejemplos. En lugar de escribir a mano las reglas de decisión, algo inviable cuando el fenómeno es complejo, se le muestra un conjunto de datos ya resueltos y el modelo ajusta sus parámetros para reproducirlos; hecho esto, sabe responder ante datos nuevos que no había visto. Cuando los ejemplos vienen etiquetados con la respuesta correcta, el aprendizaje es *supervisado*, y es el que usamos aquí. La alternativa, el aprendizaje *no supervisado*, trabaja sin etiquetas y busca agrupar los datos por semejanza, pero queda fuera de este trabajo.

Dentro del aprendizaje supervisado se distinguen dos tareas según la respuesta buscada. En **regresión** la respuesta es un número continuo: cuánto tardará un trayecto, qué precio tendrá una vivienda. En **clasificación** la respuesta es una categoría de un conjunto finito, y cuando ese conjunto tiene solo dos valores se habla de **clasificación binaria**: decidir si un correo es spam o no, si un paciente está enfermo o sano. La Figura 1.1 contrasta ambas. Nuestro problema es de clasificación binaria: decidir si una respuesta contiene una alucinación o no.

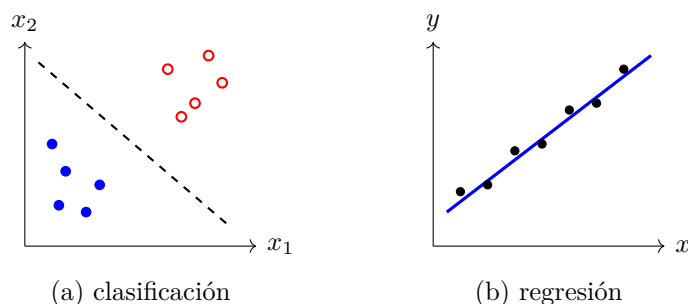


Figura 1.1: En clasificación la salida es una categoría y el modelo busca una frontera entre clases; en regresión la salida es continua y el modelo busca la tendencia.

Definición 1. Cada ejemplo se describe por un vector de *características* $x = (x_1, \dots, x_d) \in \mathbb{R}^d$, y lleva una *etiqueta* o clase y . En clasificación binaria, $y \in \{0, 1\}$.

En nuestro caso x recoge medidas de la relación entre una respuesta y su fuente, e y vale 1 si la respuesta contiene una alucinación y 0 si no. El objetivo es estimar una función f que, dado un x nuevo, prediga su clase. La mayoría de clasificadores binarios no devuelven la clase directamente, sino una probabilidad, la respuesta alucina con probabilidad 0,82; un umbral convierte esa probabilidad en una decisión, y cuanto más caro sea equivocarse, más exigente se pone.

Ajustar el modelo consiste en fijar sus parámetros con un subconjunto de *entrenamiento*. Además de esos parámetros, cada método tiene *hiperparámetros*: opciones que no se aprenden de los datos y que gobiernan su comportamiento, como la profundidad de un árbol o el número de vecinos. El modelo se juzga por cómo predice sobre datos reservados que no ha visto, el conjunto de *test*: lo que importa es **generalizar**, acertar sobre lo nuevo más allá de los ejemplos aprendidos.

El obstáculo para generalizar es el **sobreajuste**: un modelo con demasiada capacidad memoriza el ruido del entrenamiento, se ajusta a sus casualidades y falla fuera de él. El fenómeno se entiende por el compromiso **sesgo-varianza**. El *sesgo* mide cuánto se aleja el modelo de la realidad por ser demasiado rígido: uno muy simple no captura la estructura de los datos y erra sistemáticamente, tanto en entrenamiento como en test. La *varianza* mide cuánto cambia el modelo al cambiar el conjunto de entrenamiento: uno muy flexible se pliega a cada fluctuación y es inestable, brillante en entrenamiento y pobre en test. Reducir uno suele aumentar el otro, y el buen rendimiento está en el punto intermedio, al que se llega ajustando los hiperparámetros y comprobándolo con las técnicas de validación de la Sección 1.3.3.

Ningún modelo es el mejor para todo problema. La forma de la frontera que separa las clases, el número de características o el tamaño de los datos hacen que unos métodos rindan mejor que otros según el caso: un clasificador lineal fracasa si las clases no se pueden separar por una recta, mientras que uno muy flexible sobreajusta con pocos datos. Por eso en la práctica se entrenan varios y se compara su rendimiento, que es justo lo que hace el capítulo de resultados.

1.2. Text mining: del texto a números

Los modelos operan sobre vectores numéricos, pero el dato de partida es texto. La minería de textos (*text mining*) es el conjunto de técnicas que transforma un texto en una representación numérica que un algoritmo pueda procesar. Esta sección construye esa representación paso a paso, desde partir el texto en palabras hasta medir cuánto se parecen dos textos.

Tokenización y normalización. El primer paso parte el texto en unidades llamadas *tokens*. Una tokenización simple pasa el texto a minúsculas y extrae las secuencias alfa-numéricas como tokens, de modo que “WiFi” y “wifi” cuenten como la misma palabra y la puntuación se descarte. Sobre los tokens se definen los *n-gramas*: secuencias de n tokens consecutivos. Un unigrama es una palabra suelta; un bigrama, dos seguidas (“free wifi”); un trigramas, tres. Los *n-gramas* capturan combinaciones que una palabra aislada pierde, “no funciona” dice lo contrario que “funciona”, a costa de multiplicar el tamaño del vocabulario.

Dos técnicas reducen las variantes de una palabra a una forma común, para no tratar “jugar” y “jugando” como términos distintos. El *stemming* recorta la palabra a su raíz mediante reglas (“jugando”, “jugador” → “jug”); la *lematización* la lleva a su forma de diccionario usando su categoría gramatical (“jugando” → “jugar”), lo que es más preciso y más costoso. En este trabajo la tokenización se mantiene simple, sin stemming ni lematización, porque la señal que buscamos vive en el solape literal de palabras entre respuesta y fuente, y normalizar las formas la difuminaría.

Bolsa de palabras y TF-IDF. La representación más básica de un texto es la *bolsa de palabras*: se fija un vocabulario y cada texto se convierte en un vector con la frecuencia de cada término, ignorando el orden. Con los documentos “free wifi hotel”, “wifi hotel breakfast” y “cheap hotel”, el vocabulario tiene cinco términos y la matriz de frecuencias es:

	free	wifi	hotel	breakfast	cheap
d_1 : free wifi hotel	1	1	1	0	0
d_2 : wifi hotel breakfast	0	1	1	1	0
d_3 : cheap hotel	0	0	1	0	1

El defecto de esta representación es dar el mismo peso a una palabra vacía muy repetida (“de”, “el”) que a un término informativo raro. El esquema **TF-IDF** (*term frequency-inverse document frequency*) corrige ese sesgo ponderando cada término por su rareza en la colección: pesa mucho lo específico y poco lo frecuente. Para un término t en un documento, con N documentos en la colección y $df(t)$ documentos que contienen t ,

$$w_t = \underbrace{tf(t)}_{\text{frecuencia en el documento}} \cdot \underbrace{\log \frac{N}{df(t)}}_{\text{rareza en la colección}}. \quad (1.1)$$

El primer factor sube con las veces que el término aparece en el documento; el segundo lo penaliza si además aparece en muchos documentos. Un término presente en casi todos tiene $df(t) \approx N$, su logaritmo tiende a cero y su peso se anula. Sobre el ejemplo anterior, el factor de rareza es:

término	$df(t)$	$\log(N/df(t))$
hotel	3	$\log(3/3) = 0$
wifi	2	$\log(3/2) \approx 0,18$
free, breakfast, cheap	1	$\log(3/1) \approx 1,10$

La palabra “hotel”, presente en los tres documentos, recibe peso cero: es común y no distingue nada. Las que solo salen en uno pesan lo máximo, porque son las que lo caracterizan frente al resto. Así, la fila de d_1 en TF-IDF deja de ser (1, 1, 1, 0, 0) y pasa a (1,10, 0,18, 0, 0, 0): “free” manda, “hotel” desaparece.

Similitud entre dos textos. La señal central del detector es cuánto se apoya una respuesta en su fuente. Para medirlo representamos cada texto por su conjunto de palabras únicas.

Definición 2. Dada una respuesta y su fuente, sea R el conjunto de palabras únicas de la respuesta y F el de la fuente.

Sobre R y F se definen tres medidas de solape con lecturas distintas:

$$\text{containment} = \frac{|R \cap F|}{|R|}, \quad \text{Jaccard} = \frac{|R \cap F|}{|R \cup F|}, \quad \cos(u, v) = \frac{\langle u, v \rangle}{\|u\| \|v\|}. \quad (1.2)$$

El **containment** es la fracción de palabras de la respuesta que también están en la fuente; mira solo hacia la respuesta, así que no penaliza que la fuente sea larga. El **índice de Jaccard** es el solape simétrico, normalizado por la unión, y sí baja cuando los textos difieren mucho en tamaño. El **coseno TF-IDF** no trabaja sobre conjuntos, sino sobre los vectores TF-IDF de ambos textos, y mide el coseno del ángulo que forman (Figura 1.2): vale 1 si apuntan en la misma dirección, 0 si son perpendiculares, y pondera cada palabra por su rareza. Con $R = \{\text{free, wifi, indian}\}$ y $F = \{\text{wifi, indian, vegan}\}$ la intersección tiene dos palabras y la unión cuatro, luego el containment es $2/3 \approx 0,67$ y el Jaccard $2/4 = 0,50$.

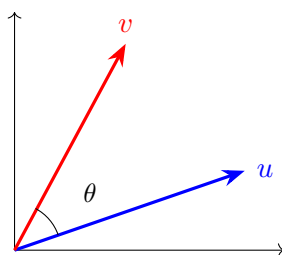


Figura 1.2: El coseno mide la similitud por el ángulo θ entre los vectores de dos textos, no por su longitud: dos textos con las mismas proporciones de palabras apuntan en la misma dirección.

El containment cae cuando la respuesta introduce palabras nuevas, ausentes de la fuente, que es la huella de una invención léxica: es la medida que más nos interesa, porque captura directamente el anclaje de la respuesta en su fuente.

Representación semántica. El solape de palabras no capta sinónimos ni paráfrasis: “coche” y “automóvil” cuentan como términos sin relación, aunque signifiquen lo mismo. Una representación que sí los acerca, sin recurrir a redes neuronales, es el *análisis semántico latente* (LSA). Consiste en aplicar una descomposición en valores singulares truncada (TruncatedSVD) a la matriz TF-IDF: esa factorización descompone la matriz en unos pocos ejes que concentran la mayor variación, y proyectar los textos sobre ellos los lleva a un espacio de pocas dimensiones donde los términos que suelen aparecer juntos quedan cerca. Es la única representación semántica que usamos, coherente con un enfoque clásico y barato.

1.3. Metodología

1.3.1. Fases del análisis

Un problema de clasificación con datos tabulares sigue un flujo estándar, del dato en crudo a la evaluación final (Figura 1.3).

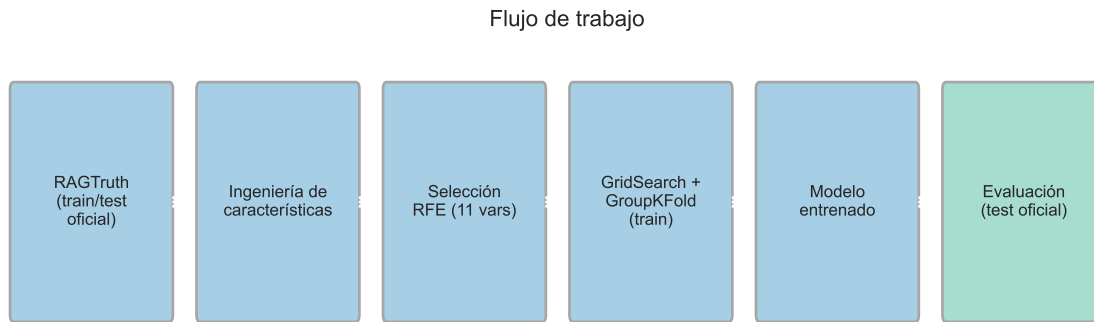


Figura 1.3: Flujo de trabajo: de los datos en crudo a la evaluación sobre el conjunto de test.

- **Análisis descriptivo.** Caracteriza los datos antes de modelar: la distribución de cada variable, el reparto de la etiqueta y las relaciones entre unas y otra. Orienta todas las decisiones posteriores y detecta problemas, como una clase muy minoritaria o variables redundantes entre sí.
- **Muestreo.** Corrige, si hace falta, un reparto de clases muy desigual mediante remuestreo: replicar la clase minoritaria (*oversampling*) o reducir la mayoritaria (*undersampling*), para que el modelo no se limite a predecir siempre la clase dominante.
- **Tratamiento de datos.** Depura las entradas: imputa los valores ausentes (*missing*) con un valor plausible y acota los valores atípicos (*outliers*) que distorsionan el ajuste.
- **Modelización.** Entrena los clasificadores y ajusta sus hiperparámetros.
- **Validación.** Estima el rendimiento sobre datos no vistos y lo cuantifica con métricas.

Las dos últimas fases concentran la carga metodológica y se desarrollan a continuación: primero las métricas con que se mide un clasificador, luego cómo se valida y se selecciona, y por último los modelos que entrenamos.

1.3.2. Métricas de evaluación

Evaluar un clasificador binario empieza por contar sus aciertos y errores. Tomando como clase positiva la que interesa detectar (aquí, la alucinación), cada predicción cae en una de cuatro casillas: los verdaderos positivos (VP) y verdaderos negativos (VN) son aciertos; los falsos positivos (FP, error de tipo I) y falsos negativos (FN, error de tipo II) son los dos tipos de fallo. Su recuento se organiza en la *matriz de confusión* (Figura 1.4).

		Predicción	
		limpia	alucina
Realidad	alucina	VP (alucina bien detectada)	FP (falsa alarma) Error tipo I
	limpia	FN (alucinación no vista) Error tipo II	VN (limpia bien detectada)

Figura 1.4: Matriz de confusión. El falso negativo (tipo II) es un positivo real que se predice negativo; el falso positivo (tipo I), un negativo que se predice positivo.

Los dos errores rara vez cuestan lo mismo, y su coste depende del problema:

	Diagnóstico médico	Detección de alucinaciones
Falso negativo	se dice sano a un paciente enfermo	una invención pasa sin ser detectada
Falso positivo	se trata a un paciente sano	se marca como inventado un texto correcto

En nuestro caso el falso negativo, una alucinación que se cuela, es el error caro, y las métricas siguientes permiten primar su detección.

Sobre los cuatro conteos se definen las dos medidas básicas. La **precisión** indica cuántas de las predicciones positivas fueron correctas, es decir, cómo de fiable es el modelo cuando marca algo como positivo. La **exhaustividad** o *recall* (también sensibilidad) indica cuántos de los positivos reales llegó a detectar:

$$\text{precisión} = \frac{VP}{VP + FP}, \quad \text{recall} = \frac{VP}{VP + FN}. \quad (1.3)$$

Precisión y recall se mueven en sentidos opuestos, y el equilibrio lo fija el umbral de decisión. Si el modelo solo predice positivo cuando está muy seguro, acierta casi siempre que lo hace (precisión alta), pero deja escapar muchos positivos dudosos (recall bajo). Si predice positivo a la mínima, no se le escapa casi ninguno (recall alto) a costa de muchas falsas alarmas (precisión baja). En el extremo, un modelo que marca todo como positivo tiene recall 1 y precisión igual a la simple proporción de positivos.

El valor **F1** resume ambos en su media armónica, que solo es alta cuando los dos lo son (si uno se acerca a cero, arrastra al conjunto):

$$F_1 = 2 \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}. \quad (1.4)$$

El F1 supone que precisión y recall importan igual, lo que no siempre ocurre. Cuando detectar positivos pesa más que evitar falsas alarmas, se usa su generalización F_β , que valora el recall β^2 veces más que la precisión:

$$F_\beta = (1 + \beta^2) \frac{\text{precisión} \cdot \text{recall}}{\beta^2 \text{precisión} + \text{recall}}. \quad (1.5)$$

Una medida más intuitiva, la **accuracy** o fracción de aciertos, $\frac{VP+VN}{VP+VN+FP+FN}$, engaña cuando una clase domina: si el 90 % de los ejemplos son negativos, predecir siempre negativo ya da un 90 % de accuracy sin detectar un solo positivo. La **balanced accuracy** corrige ese espejismo promediando el recall de cada clase. Un ejemplo lo fija: sobre 300 respuestas con 100 alucinaciones reales, un detector caza 70 ($VP = 70$, $FN = 30$) a costa de 20 falsas alarmas ($FP = 20$, $VN = 180$). Su precisión es $70/90 \approx 0,78$, el recall $70/100 = 0,70$ y el $F_1 \approx 0,74$; la accuracy sube a $(70+180)/300 \approx 0,83$ porque acierta casi todos los negativos, que son mayoría, mientras que la balanced accuracy, $\frac{1}{2}(0,70 + 0,90) = 0,80$, rebaja esa cifra optimista.

Todas las métricas anteriores dependen del umbral con que la probabilidad se convierte en decisión. Dos curvas recorren todos los umbrales de golpe. La **curva ROC** (Figura 1.5, izquierda) contrapone la tasa de verdaderos positivos (el recall) frente a la tasa de falsos positivos al variar el umbral; su área, el **AUC**, es la probabilidad de que un positivo tomado al azar reciba mayor score que un negativo tomado al azar. El AUC tiene dos propiedades que lo hacen una medida sólida para comparar modelos: es invariante a la escala del score, pues solo importa el orden de las predicciones, y es invariante al umbral, pues los resume todos. La **curva precisión–recall** (Figura 1.5, derecha), en cambio, sí refleja el desbalance de clases, porque la precisión depende de la proporción de positivos; un modelo bueno mantiene la precisión alta aun a recall alto, y su curva se acerca a la esquina superior derecha. Por eso la miramos además del AUC cuando la clase positiva es minoritaria.

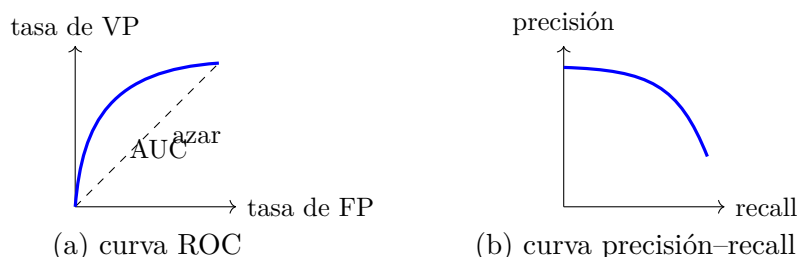


Figura 1.5: Las dos curvas que recorren todos los umbrales. La ROC resume la separación en el AUC; la precisión–recall muestra a partir de qué recall se degrada la precisión.

Elegido un modelo, queda fijar el umbral t que decide positivo cuando el score supera t . El **índice de Youden** elige el que maximiza $J = \text{sensibilidad} + \text{especificidad} - 1$, esto es, el punto de la curva ROC más alejado de la diagonal, donde mejor se separan las dos clases.

1.3.3. Validación y selección de características

Validación cruzada. Medir el rendimiento sobre los mismos datos con que se entrena infla el resultado, porque el modelo ya los conoce. Reservar una parte para test lo evita, pero entonces el resultado depende de qué ejemplos tocaron en esa partición. La **validación cruzada** de k iteraciones elimina esa dependencia: parte el entrenamiento en k bloques, entrena con $k - 1$ y valida con el restante, y repite rotando el bloque de validación (Figura 1.6); el rendimiento final es el promedio de las k rondas, que ya no depende de una partición afortunada.

Esta partición debe respetar la estructura de los datos. Cuando varios ejemplos comparten una misma unidad, aquí un mismo documento fuente del que salen varias respuestas,

esa unidad tiene que caer entera en un solo bloque. Si no, el modelo ve en validación una respuesta casi idéntica a otra que tuvo en entrenamiento y el resultado se infla por esa filtración. La variante que lo garantiza es el **GroupKFold**, que agrupa por la unidad e impide que un documento aparezca a la vez en entrenamiento y validación.

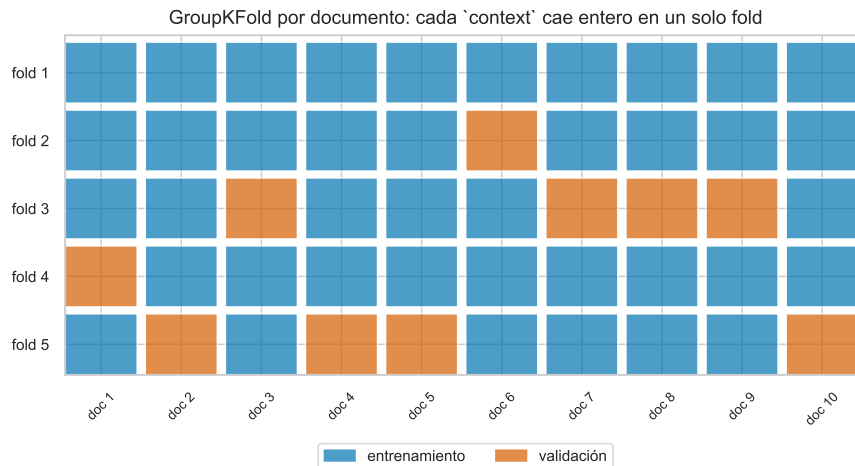


Figura 1.6: GroupKFold por documento: cada fuente cae entera en un único bloque, de modo que ninguna aparece a la vez en entrenamiento (azul) y validación (naranja).

Búsqueda en rejilla. Casi todos los modelos tienen hiperparámetros, y distintas combinaciones dan resultados distintos: no es lo mismo tomar los 3 vecinos más cercanos con distancia euclídea que los 11 con distancia de Manhattan. La **búsqueda en rejilla** (*grid search*) los ajusta probando un conjunto de combinaciones, evaluando cada una por validación cruzada y conservando la mejor, sin tocar el test (Figura 1.7). Su coste crece con el producto de los valores de cada hiperparámetro por las iteraciones de la validación: optimizar tres hiperparámetros con tres valores cada uno y validación cruzada de cuatro iteraciones ya son $3 \cdot 3 \cdot 3 \cdot 4 = 108$ ajustes. Para modelos clásicos ese coste es asumible en un ordenador corriente.

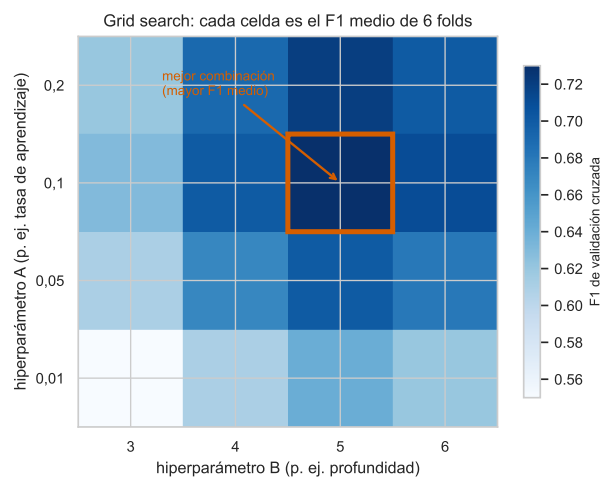


Figura 1.7: Búsqueda en rejilla: cada celda es el rendimiento medio de una combinación de hiperparámetros en validación cruzada; se conserva la mejor.

Selección de características. Con muchas variables conviene quedarse con las informativas y descartar las redundantes, tanto para mejorar el modelo como para entenderlo. Hay dos familias. Los métodos de **filtro** puntúan cada variable por una medida estadística de su relación con la etiqueta, al margen del modelo; son rápidos, pero ignoran las interacciones y la redundancia entre variables, así que pueden quedarse con dos variables casi idénticas. Los métodos **envolventes** (*wrapper*) usan el propio modelo como juez: prueban subconjuntos y se quedan con el que mejor valida, a mayor coste. La **eliminación recursiva de características** (RFE) es el envoltente que usamos: entrena con todas las variables, elimina la menos importante, reentrena y repite, lo que produce un orden de importancia. El número final de variables se fija por el *método del codo*, el punto de la curva de rendimiento a partir del cual añadir variables ya no aporta.

1.3.4. Regresión logística

La regresión logística es el clasificador lineal por excelencia y sirve de **modelo base**: la referencia simple e interpretable contra la que se miden los demás. Pese a su nombre, resuelve una clasificación, no una regresión. La idea es modelar la probabilidad de la clase positiva como una combinación lineal de las características, pero acotada al intervalo $(0, 1)$ para que pueda leerse como probabilidad. El acotado lo hace la función *sigmoide*:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^\top x, \quad p(y = 1 | x) = \sigma(z). \quad (1.6)$$

La sigmoide (Figura 1.8) comprime cualquier valor real a $(0, 1)$: valores muy negativos de z dan probabilidad cercana a 0, muy positivos cercana a 1, y $z = 0$ da exactamente 0,5, que marca la frontera de decisión.

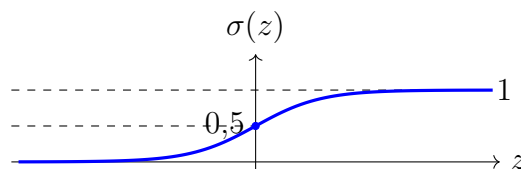


Figura 1.8: Función sigmoide: transforma la combinación lineal z en una probabilidad entre 0 y 1.

La sigmoide admite una lectura muy útil si se despeja. Partiendo de $p = \sigma(z) = 1/(1 + e^{-z})$ se tiene $1 - p = e^{-z}/(1 + e^{-z})$, y dividiendo,

$$\frac{p}{1 - p} = e^z \implies \log \frac{p}{1 - p} = z = \beta_0 + \beta^\top x. \quad (1.7)$$

Es decir, la combinación lineal es el logaritmo de la razón entre la probabilidad de la clase positiva y la de la negativa, el *log-odds*. Esta forma da a los coeficientes una lectura directa, y es la mayor virtud del modelo: cada β_i es cuánto varía el log-odds de la clase positiva al aumentar una unidad la característica i , con las demás fijas. Su signo indica el sentido (un β_i positivo empuja hacia la clase positiva) y su magnitud, la fuerza. Un coeficiente negativo grande sobre el containment, por ejemplo, significaría que a mayor solape con la fuente, menor probabilidad de alucinación, justo lo esperable.

Para ver el modelo en acción conviene reducirlo a una sola característica. Supongamos que clasificamos respuestas usando únicamente su containment x , con $p(x) = \sigma(\beta_0 + \beta_1 x)$.

Cada respuesta del entrenamiento se sitúa en el eje según su containment, y a partir de las etiquetas se estiman β_0 y β_1 , que dibujan una sigmoide: las respuestas con poco solape reciben probabilidad alta de alucinar y las de mucho solape, baja. Para clasificar una respuesta nueva basta situarla por su containment y leer la probabilidad que le asigna la curva; si supera el umbral, se marca como alucinación.

Los coeficientes se estiman buscando los que hacen más probables las etiquetas observadas, lo que equivale a minimizar la *entropía cruzada*, una función de pérdida que castiga predecir con confianza la clase equivocada:

$$\mathcal{L}(\beta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (1.8)$$

Cuando la etiqueta real es $y_i = 1$, solo pesa el término $-\log p_i$, que se dispara si el modelo asigna a ese ejemplo una probabilidad p_i próxima a cero; cuando es $y_i = 0$, pesa $-\log(1 - p_i)$, con el castigo simétrico. Esta pérdida es convexa, y su mínimo se busca por descenso de gradiente: se parte de unos coeficientes cualesquiera y se corrigen paso a paso en la dirección que más reduce la pérdida, hasta converger. Es el mismo método que está en la base de muchos algoritmos de aprendizaje. Para frenar el sobreajuste se añade un término de **regularización** que penaliza coeficientes grandes: la penalización L2 los encoge de forma suave, y la L1 lleva algunos a cero exacto, con lo que además descarta variables.

- **Ventajas:** interpretable por sus coeficientes, muy barata de entrenar y con probabilidades bien calibradas.
- **Desventajas:** solo traza fronteras lineales, así que se queda corta cuando la separación entre clases es curva; conviene además quitar antes las variables muy correlacionadas entre sí.

1.3.5. K vecinos más cercanos

El clasificador de k vecinos más cercanos (KNN) no construye ningún modelo explícito: guarda los datos de entrenamiento y clasifica cada ejemplo nuevo por **vecindad**, mirando a qué clase pertenecen los que tiene alrededor. La idea plantea dos preguntas: dónde se sitúan los datos y cómo se mide la cercanía. Lo primero se resuelve con el vector de características, que ubica cada ejemplo como un punto de $\mathbb{R}^d$. Para lo segundo se usa una distancia; las más habituales son la euclídea (la del teorema de Pitágoras) y la de Manhattan (la suma de diferencias en valor absoluto, como recorrer una cuadrícula de calles),

$$d_{\text{euclídea}}(p, q) = \sqrt{\sum_i (p_i - q_i)^2}, \quad d_{\text{Manhattan}}(p, q) = \sum_i |p_i - q_i|. \quad (1.9)$$

Para clasificar un punto nuevo se toman sus k vecinos más cercanos y se le asigna la clase mayoritaria entre ellos (Figura 1.9). La elección de k cambia el resultado: en la figura, el punto nuevo tiene dos vecinos de una clase y uno de otra entre los tres más próximos, así que con $k = 3$ se clasifica en la primera; ampliando a $k = 5$ entran vecinos más lejanos que pueden invertir el voto.

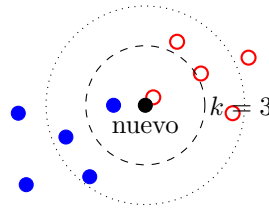


Figura 1.9: KNN: el punto nuevo (negro) se clasifica por la clase mayoritaria entre sus vecinos. El círculo interior es $k = 3$; ampliar el radio ($k = 5$) puede cambiar la decisión.

El parámetro k gobierna el compromiso sesgo–varianza: un k pequeño sigue el ruido local y sobreajusta, mientras que uno grande promedia sobre vecinos lejanos y aumenta el sesgo, difuminando la frontera entre clases parecidas. El mejor valor depende de los datos y se busca por validación, aunque una regla común de partida es $k = \sqrt{N}$, con N el número de ejemplos. Como todo se reduce a distancias, KNN es sensible a la escala de las variables, así que se estandarizan antes; sin ello, una característica de rango amplio dominaría el cálculo y ahogaría a las demás.

- **Ventajas:** no asume ninguna forma para la frontera entre clases, funciona bien con muchos datos y no requiere una fase de entrenamiento.
- **Desventajas:** clasificar es costoso, pues mide la distancia del punto nuevo a todo el entrenamiento, y el método se degrada cuando hay muchas dimensiones o la distancia está mal elegida.

1.3.6. Árboles de decisión

Un árbol de decisión clasifica mediante una secuencia de preguntas, y es la pieza sobre la que se construyen los dos modelos siguientes. Tiene dos elementos: los *nodos* internos, donde se toma una decisión partiendo los datos por un umbral sobre una característica (“¿containment < 0,5?”), y las *hojas*, que asignan una clase (Figura 1.10). Clasificar un ejemplo es recorrer el árbol desde la raíz, siguiendo en cada nodo la rama que le corresponde, hasta llegar a una hoja.

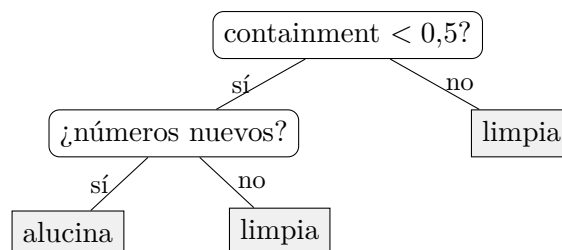


Figura 1.10: Árbol de decisión: los nodos parten los datos por un umbral; las hojas asignan la clase.

El árbol se construye de arriba abajo y de forma recursiva. En cada nodo prueba cortes sobre las características y se queda con el que más reduce la *impureza* de los nodos que genera, es decir, el que deja a cada lado ejemplos lo más de una sola clase posible. La impureza de un nodo con proporciones de clase p_k se mide con el índice de Gini o con la entropía:

$$G = 1 - \sum_k p_k^2, \quad H = - \sum_k p_k \log_2 p_k. \quad (1.10)$$

Ambas valen cero cuando el nodo es puro (una sola clase) y son máximas cuando las clases están al 50 %. Por ejemplo, un nodo con 40 ejemplos que alucinan y 60 limpios tiene $G = 1 - 0,4^2 - 0,6^2 = 0,48$; un corte que lo separe en un nodo de 30 alucina y 10 limpia ($G = 0,375$) y otro de 10 alucina y 50 limpia ($G \approx 0,28$) baja la impureza media ponderada por debajo de 0,48, así que el árbol prefiere ese corte al que menos la reduzca. El proceso se repite en cada nodo y se detiene cuando el nodo es puro, no quedan características útiles o se alcanza un límite fijado de antemano.

Sin freno, el árbol crece hasta separar por completo el entrenamiento, memorizándolo, lo que lo hace muy flexible pero propenso al sobreajuste: un árbol profundo aprende las casualidades de los datos y falla con datos nuevos. La profundidad máxima y el número mínimo de ejemplos por hoja son los hiperparámetros que controlan ese crecimiento.

- **Ventajas:** captura interacciones entre variables, es insensible a la escala, maneja bien características irrelevantes y su lógica es legible como una cadena de reglas.
- **Desventajas:** un árbol único es inestable, pues pequeños cambios en los datos dan árboles muy distintos, y sobreajusta con facilidad. Los dos modelos siguientes corrigen esto combinando muchos árboles.

1.3.7. Random Forest

Un bosque aleatorio (*Random Forest*) combina muchos árboles para ganar la estabilidad que a uno solo le falta. Se apoya en una técnica llamada *bagging* (*bootstrap aggregating*): de la base de entrenamiento se extraen B muestras aleatorias con reemplazo, cada una del mismo tamaño que la original pero con ejemplos repetidos y otros ausentes, y con cada muestra se entrena un árbol. Para clasificar un ejemplo nuevo, cada uno de los B árboles emite su voto y se toma la clase más votada,

$$\hat{y}(x) = \text{clase más votada entre } \{T_1(x), \dots, T_B(x)\}. \quad (1.11)$$

El bagging reduce la varianza sin aumentar el sesgo: la predicción de un solo árbol es muy sensible al ruido del entrenamiento, pero el promedio de muchos árboles poco correlacionados cancela ese ruido individual, igual que promediar muchas mediciones ruidosas se acerca al valor real. Para que los árboles no se parezcan demasiado, y su promedio valga la pena, el bosque añade un segundo ingrediente de azar: en cada corte no considera todas las características, sino un subconjunto aleatorio de ellas (típicamente $\sqrt{d}$ de las d disponibles en clasificación). Así ningún árbol se apoya siempre en la misma variable dominante, y el conjunto queda des-correlacionado. El número de árboles B , su profundidad y cuántas características se consideran en cada corte son sus hiperparámetros, y suelen buscarse probando.

- **Ventajas:** robusto y preciso, uno de los mejores clasificadores para datos tabulares; hereda del árbol la captura de interacciones y la insensibilidad a la escala, maneja muchas variables y apenas sobreajusta.
- **Desventajas:** pierde la interpretabilidad de un árbol único, es más costoso de entrenar y evaluar, y puede desajustarse con datos muy ruidosos si no se controla el número de árboles.

1.3.8. Gradient boosting

El *gradient boosting* también combina árboles, pero de forma **secuencial** en vez de paralela. En lugar de promediar árboles independientes, construye el modelo por etapas:

empieza con una predicción sencilla y, en cada etapa, añade un árbol nuevo entrenado para corregir el error que dejan los anteriores. Ese error a corregir es el gradiente de la función de pérdida, de ahí el nombre. El modelo tras m etapas es la suma del anterior más el árbol nuevo,

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad (1.12)$$

donde h_m es el árbol de la etapa m y $\nu \in (0, 1]$ es la *tasa de aprendizaje*, que modera cuánto contribuye cada árbol. Una tasa pequeña avanza despacio pero generaliza mejor; una grande, o demasiados árboles, sobreajustan, así que tasa y número de árboles se ajustan juntos.

La diferencia con el bosque aleatorio es de fondo. El bosque promedia árboles independientes para reducir la *varianza*, partiendo de árboles que ya de por sí aciertan; el boosting encadena árboles dependientes, cada uno atacando lo que el anterior no supo, para reducir el *sesgo*, partiendo de un modelo débil que va mejorando. Eso suele dar mayor precisión, a cambio de un ajuste más delicado y más lento. **XGBoost** es una implementación eficiente y regularizada de esta idea, muy usada en la práctica, y es el clasificador que mejor rinde en el caso práctico.

- **Ventajas:** suele ofrecer la mayor precisión de los modelos clásicos sobre datos tabulares y maneja bien las interacciones entre variables.
- **Desventajas:** tiene más hiperparámetros que ajustar, entrena más despacio que un bosque y su riesgo de sobreajuste es mayor si no se regula con cuidado.